

PRODUCT DOCUMENTATION

Unit Converter

User guide & technical documentation — convert length, weight, volume, temperature, area, currency and digital storage instantly in the browser, with exact international factors, live currency rates when online, and an honestly labelled offline fallback.

Creator	**Dr. Asif Ahmad**
Product	Unit Converter — version 1.0
Category	Productivity
Published by	Asif SaaS Studio
Date	20 October 2026
Format	Single-file web application (index.html, 17.4 KB)

Private by design: every value you type stays on your device — no uploads, no accounts, no tracking. The only network call the app can ever make is the optional currency-rate refresh (Chapter 4).

Contents

1. What this product does
2. The seven categories and their units
3. How conversion works
4. Currency — live rates and the honest fallback
5. Input and validation
6. Interface walkthrough
7. How correctness was verified
8. Uploading to Asif SaaS Studio (Wix CMS)
9. Limits and honest notes
10. Product information and credits
11. Appendix — the tested core, verbatim

1. What this product does

Unit Converter turns one number into its equivalent in every related unit, instantly, as you type. Pick a category, choose the from and to units, type or paste a value — the result, the exact conversion factor, and a table showing the value in **every** unit of the category appear immediately. Seven categories cover everyday, scientific, kitchen, land-measurement and computing needs, including Pakistani land units (marla and kanal) that most converters omit, and a currency category whose rates refresh live from a free public source whenever the device is online.

Feature	What it gives you
7 categories · 66 units	Length, Weight, Volume, Temperature, Area, Currency (20 currencies), Digital storage.
Instant conversion	Result updates on every keystroke; a factor line states "1 unit = X units" for the chosen pair.
"In every unit" table	The entered value converted into all units of the category at once.
Swap & Clear buttons	One tap reverses the direction; one tap empties the field.
Copy result	One tap copies the full result line to the clipboard.
Live currency rates	Fetched keyless from open.er-api.com on load and on demand; dated indicative snapshot when offline.

2. The seven categories and their units

Each linear category converts through a base unit. The factor shown is the number of base units in one unit of that row (value in base = value × factor).

2.1 Length — base unit: metre (m)

Unit	Symbol	Factor (metres)
Millimetre	mm	0.001
Centimetre	cm	0.01
Metre	m	1
Kilometre	km	1,000
Inch	in	0.0254 (exact, by definition)
Foot	ft	0.3048 (exact)
Yard	yd	0.9144 (exact)
Mile	mi	1,609.344 (exact)

2.2 Weight — base unit: kilogram (kg)

Unit	Symbol	Factor (kilograms)
Milligram	mg	0.000001
Gram	g	0.001
Kilogram	kg	1
Metric tonne	t	1,000
Ounce	oz	0.028349523125 (exact)
Pound	lb	0.45359237 (exact, international avoirdupois pound)
Stone	st	6.35029318 (= 14 lb)

2.3 Volume — base unit: litre (L)

Unit	Symbol	Factor (litres)
Millilitre	mL	0.001
Litre	L	1
Cubic metre	m³	1,000
Teaspoon (US)	tsp	0.00492892159375
Tablespoon (US)	tbsp	0.01478676478125
Fluid ounce (US)	fl oz	0.0295735295625
Cup (US)	cup	0.2365882365
Pint (US)	pt	0.473176473
Gallon (US)	gal	3.785411784 (exact)
Gallon (UK)	gal UK	4.54609

2.4 Temperature — formulas, not factors

Temperature scales have different zero points, so they cannot be converted by a factor. The app uses the exact definitions: `°F = °C × 9/5 + 32` · `°C = (°F − 32) × 5/9` · `K = °C + 273.15`. Units: Celsius (°C), Fahrenheit (°F), Kelvin (K).

2.5 Area — base unit: square metre (m²)

Unit	Symbol	Factor (m²)
Square metre	m²	1
Square foot	ft²	0.09290304 (exact)
Square yard	yd²	0.83612736
Acre	acre	4,046.8564224
Hectare	ha	10,000
Marla	marla	25.29285264 (= 272.25 ft²)
Kanal	kanal	505.8570528 (= 20 marla)

2.6 Currency — 20 currencies, through USD

US Dollar (USD), Euro (EUR), British Pound (GBP), Pakistani Rupee (PKR), Indian Rupee (INR), Bangladeshi Taka (BDT), Chinese Yuan (CNY), UAE Dirham (AED), Saudi Riyal (SAR), Qatari Riyal (QAR), Kuwaiti Dinar (KWD), Canadian Dollar (CAD), Australian Dollar (AUD), Japanese Yen (JPY), Swiss Franc (CHF), Singapore Dollar (SGD), Malaysian Ringgit (MYR), Turkish Lira

(TRY), South African Rand (ZAR), Nigerian Naira (NGN). Rates are stored as units per 1 USD and every conversion passes through USD: `result = value ÷ rate[from] × rate[to]`. See Chapter 4 for where the rates come from.

2.7 Digital storage — base unit: byte (B)

Unit	Symbol	Factor (bytes)
Bit	bit	0.125 (= ⅛ byte)
Byte	B	1
Kilobyte (decimal)	KB	1,000
Megabyte (decimal)	MB	1,000,000
Gigabyte (decimal)	GB	1,000,000,000
Terabyte (decimal)	TB	1,000,000,000,000
Petabyte (decimal)	PB	1,000,000,000,000,000
Kibibyte (binary)	KiB	1,024
Mebibyte (binary)	MiB	1,048,576
Gibibyte (binary)	GiB	1,073,741,824
Tebibyte (binary)	TiB	1,099,511,627,776

> **SI vs IEC:** both families are included and labelled, because they are genuinely different sizes. KB/MB/GB (SI, powers of 1,000) are what disk makers and network speeds use; KiB/MiB/GiB (IEC, powers of 1,024) are what operating systems usually mean. 1 GiB = 1,024 MiB ≈ 1.0737 GB — the app shows both so the two are never confused.

3. How conversion works

For every linear category the engine holds one exact factor per unit, relative to a base unit (Chapter 2). Converting is two steps: `base = value × factor[from]`, then `result = base ÷ factor[to]`. Because both directions use the same stored factors, converting A → B → A returns the starting value exactly (verified by the round-trip sweeps in Chapter 7). Where a factor is an international definition — the inch at exactly 25.4 mm, the pound at exactly 0.45359237 kg, the US gallon at exactly 3.785411784 L — the full defining value is stored, so results carry no rounding drift from the factors themselves.

Temperature is the exception: it is affine (the scales disagree about zero, not just about step size), so it is converted by the formulas in §2.4 through Celsius. Currency is the second exception: it converts through USD using the current rate table (Chapter 4). Results are

displayed to 10 significant figures with trailing zeros trimmed; exponential notation appears only for extreme magnitudes (beyond 10^{15} or below 10^{-9}).

4. Currency — live rates and the honest fallback

Exchange rates change every day, so no rate table printed inside a file can stay "live". This app is honest about that, in three parts:

- **Live when online.** On load — and whenever the Refresh rates button is pressed — the app fetches the latest rates from `open.er-api.com`, a free public service that needs **no account, no API key and no backend**. When the fetch succeeds, the status line reads "Rates: LIVE from open.er-api.com" with the source's own update time, and all conversions use those rates.
- **A dated fallback, never disguised.** If the fetch fails — no internet, or the host page blocks network calls — the app falls back to a built-in snapshot captured on **20 Oct 2026** (for example 1 USD = 276.779576 PKR, 0.891477 EUR, 0.751092 GBP on that date). The status line then says "built-in indicative snapshot, 20 Oct 2026", and currency factor lines are tagged "(indicative rate)". The snapshot is never labelled live.
- **No backend was needed — and its limits are stated.** Because the rate source is keyless and public, this browser-only design is complete as shipped; nothing was faked. A commercial feed with guaranteed rates (the kind used for trade settlement) would be a different architecture: a paid provider key held on a small backend proxy that the page calls over HTTPS. That is deliberately out of scope, and the app states in its own footer that it is not for legal or trade settlement use.

5. Input and validation

The value field is deliberately forgiving about how numbers are pasted, and strict about what a number is:

Accepted	Examples
Plain and decimal numbers	`42` · `3.25` · `.5`
Thousands separators and spaces	`1,234.5` · `42 ` (commas/spaces are stripped)
Signs and scientific notation	`-3` · `1e3`

Rejected	What happens
`abc` , `1.2.3` , `NaN` , `Infinity`	A plain-language error appears ("Please enter a valid number…"), the field is outlined in red, and no result is shown.
Empty field	A neutral state — no error, no result. Clearing the box is a valid action, not a mistake.

6. Interface walkthrough

- **Category chips** — the seven categories across the top; the active one is filled emerald. Switching category rebuilds the unit lists and re-converts immediately (sensible defaults: Celsius → Fahrenheit for Temperature, USD → PKR for Currency).
- **From / To selectors** — full unit names with symbols, e.g. "Gallons (US) (gal)".
- **Swap button (⇅)** — the round button between the selectors exchanges From and To; on narrow phone screens it rotates and stacks between the two fields.
- **Result panel** — the dark emerald panel states the full conversion ("1 in = 2.54 cm") in large type; a muted line beneath gives the unit factor for the chosen pair.
- **Currency status bar** — appears only in the Currency category: it states whether rates are LIVE (with update time) or the dated indicative snapshot, and carries the **Refresh rates** button.
- **Copy result** — copies the result line; the button confirms with "Copied ✓". If the clipboard is blocked by the host page, it falls back to a selectable copy and otherwise leaves the result visible to copy by hand.
- **Clear** — empties the value field and returns focus to it.
- **"In every unit" table** — under the actions, the entered value expressed in every unit of the category, updating live.

7. How correctness was verified

The conversion core that ships inside `index.html` (Appendix) was extracted and exercised by an automated suite in Node: **3,208 checks, all passed, 0 failed**.

Check group	What was asserted
Exact definitions	1 in = 2.54 cm · 1 mi = 1.609344 km · 1 ft = 12 in · 1 lb = 0.45359237 kg · 16 oz = 1 lb · 1 US gal = 3.785411784 L · 1 tsp = 4.92892159375 mL · 1 cup = 16 tbsp · 1 UK gal = 4.54609 L · 1 acre = 4,046.8564224 m² · 1 kanal = 20 marla · 1 marla = 272.25 ft² · 1 GiB = 1,024 MiB · 1 KB = 1,000 B · 8 bit = 1 B
Temperature identities	0 °C = 32 °F · 100 °C = 212 °F · −40 °C = −40 °F · 0 °C = 273.15 K · 98.6 °F = 37 °C
Currency maths	Against the built-in snapshot (1 USD = 276.779576 PKR, 20 Oct 2026) and cross-rate consistency (EUR → GBP via USD).
Round-trip sweeps	Every unit pair in every category × 4 test values — converting there and back returned the starting value in all cases.
Validation set	Accepts `1,234.5`, `42`, `.5`, `-3`, `1e3`; rejects `abc`, `1.2.3`, `NaN`, `Infinity`; empty input is neutral.
Formatting	Trailing zeros trimmed; 10 significant figures; exponential form only for extreme magnitudes.

8. Uploading to Asif SaaS Studio (Wix CMS)

Step 1. Wix dashboard → CMS → **SaasProducts** → Add Item.

Step 2. Fill the listing fields:

Field	Value
Title	Unit Converter — Length, Weight, Volume, Temperature, Currency & Storage
Category	Productivity
Slug	unit-converter
Summary	Convert length, weight, volume, temperature, area (marla/kanal included), currency with live rates, and digital storage — instantly, in your browser.
Version	1.0
Demo	No
appHtml	Paste the full contents of `index.html`.

Step 3. Set **published** on, save, open the live app page and run the checklist: 1 in → 2.54 cm with the every-unit table; 100 °C → 212 °F and swap back; currency status line shows LIVE (or the dated snapshot offline) and 100 USD converts to PKR; pasting `1,234.5` works, `abc` shows the friendly error; Clear empties the field; Copy result confirms "Copied ✓"; the layout stacks neatly on a phone.

9. Limits and honest notes

- **Rates are reference-grade.** The live source updates about once a day and reflects mid-market rates, not the rate any bank or exchange house will actually give; the offline snapshot is older still and is labelled with its date. Use the app for estimates, study and everyday decisions — not for legal or trade settlement.
- **KB is not KiB.** The storage category includes both families precisely because confusing them is the most common storage error: 1 KB = 1,000 bytes; 1 KiB = 1,024 bytes (§2.7).
- **Cooking units are US customary.** Teaspoon, tablespoon, fluid ounce, cup and pint are the US measures; the US and UK gallons differ (3.785 L vs 4.546 L) and both are included and labelled.
- **Marla and kanal follow the standard Pakistani definition** used in Punjab revenue practice: 1 marla = 272.25 ft², 1 kanal = 20 marla. Local historic variants exist; for legal land work, confirm against the local record.
- **Offline behaviour.** Every category except live currency works fully offline; currency offline uses the dated snapshot and says so.
- **Privacy model.** No localStorage, cookies, accounts, analytics or uploads; no service workers, permission prompts or parent-window access — safe in a sandboxed iframe. The single possible network call is the currency-rate fetch.

10. Product information and credits

Field	Detail
Product	Unit Converter
Version	1.0 (20 October 2026)
Creator	**Dr. Asif Ahmad**
Publisher	Asif SaaS Studio
Category / slug	Productivity / unit-converter
Deliverable	index.html — 17,397 bytes, single file, all CSS and JavaScript inline
Requirements	Any modern browser; no server, no installation, no account. Internet optional (live currency rates only).
Rate source	open.er-api.com (free, keyless); built-in indicative snapshot dated 20 Oct 2026 as fallback
Verification	3,208 automated checks passed, 0 failed (Chapter 7)

Appendix — the tested core, verbatim

The code below is the complete conversion core of the product exactly as it ships inside `index.html`: the rate snapshot, the full category/unit data with factors, the temperature functions, the `convert` engine, the input parser and the number formatter. Interface wiring is omitted.

```
/* Conversion data. Linear categories convert through a base unit via factor f
   (value_in_base = value * f). Temperature is affine and handled by functions.
   Currency converts through USD using rates = units per 1 USD. */
var FX_SNAPSHOT = { USD:1, EUR:0.891477, GBP:0.751092, PKR:276.779576, INR:95.052511,
  BDT:123.584647, CNY:6.693661, AED:3.6725, SAR:3.75, QAR:3.64, KWD:0.310405,
  CAD:1.409964, AUD:1.432547, JPY:158.441511, CHF:0.826422, SGD:1.279789,
  MYR:4.082505, TRY:49.581764, ZAR:16.42731, NGN:1332.819531 };
var FX_SNAPSHOT_DATE = "20 Oct 2026";

var CATS = {
  length: { label:"Length", units:{
    mm:{n:"Millimetres", s:"mm", f:0.001}, cm:{n:"Centimetres", s:"cm", f:0.01},
    m:{n:"Metres", s:"m", f:1}, km:{n:"Kilometres", s:"km", f:1000},
    inch:{n:"Inches", s:"in", f:0.0254}, ft:{n:"Feet", s:"ft", f:0.3048},
    yd:{n:"Yards", s:"yd", f:0.9144}, mi:{n:"Miles", s:"mi", f:1609.344} } },
  weight: { label:"Weight", units:{
    mg:{n:"Milligrams", s:"mg", f:1e-6}, g:{n:"Grams", s:"g", f:1e-3},
    kg:{n:"Kilograms", s:"kg", f:1}, ton:{n:"Metric tonnes", s:"t", f:1000},
    oz:{n:"Ounces", s:"oz", f:0.028349523125}, lb:{n:"Pounds", s:"lb", f:0.45359237},
    stone:{n:"Stone", s:"st", f:6.35029318} } },
  volume: { label:"Volume", units:{
    ml:{n:"Millilitres", s:"mL", f:0.001}, l:{n:"Litres", s:"L", f:1},
    m3:{n:"Cubic metres", s:"m\u00B3", f:1000},
    tsp:{n:"Teaspoons (US)", s:"tsp", f:0.00492892159375},
    tbsp:{n:"Tablespoons (US)", s:"tbsp", f:0.01478676478125},
    floz:{n:"Fluid ounces (US)", s:"fl oz", f:0.0295735295625},
    cup:{n:"Cups (US)", s:"cup", f:0.2365882365},
    pint:{n:"Pints (US)", s:"pt", f:0.473176473},
    galUS:{n:"Gallons (US)", s:"gal", f:3.785411784},
    galUK:{n:"Gallons (UK)", s:"gal UK", f:4.54609} } },
  temperature: { label:"Temperature", temp:true, units:{
    c:{n:"Celsius", s:"\u00B0C"}, f:{n:"Fahrenheit", s:"\u00B0F"}, k:{n:"Kelvin", s:"K"} } },
  area: { label:"Area", units:{
    m2:{n:"Square metres", s:"m\u00B2", f:1}, ft2:{n:"Square feet", s:"ft\u00B2", f:0.09290304},
    yd2:{n:"Square yards", s:"yd\u00B2", f:0.83612736},
    acre:{n:"Acres", s:"acre", f:4046.8564224}, ha:{n:"Hectares", s:"ha", f:10000},
    marla:{n:"Marla", s:"marla", f:25.29285264}, kanal:{n:"Kanal", s:"kanal", f:505.8570528} } },
  storage: { label:"Digital storage", units:{
    bit:{n:"Bits", s:"bit", f:0.125}, byte:{n:"Bytes", s:"B", f:1},
    kb:{n:"Kilobytes (KB, 1000)", s:"KB", f:1000}, mb:{n:"Megabytes (MB, 1000)", s:"MB", f:1e6},
    gb:{n:"Gigabytes (GB, 1000)", s:"GB", f:1e9}, tb:{n:"Terabytes (TB, 1000)", s:"TB", f:1e12},
    pb:{n:"Petabytes (PB, 1000)", s:"PB", f:1e15},
    kib:{n:"Kibibytes (KiB, 1024)", s:"KiB", f:1024}, mib:{n:"Mebibytes (MiB, 1024)", s:"MiB", f:1048576},
    gib:{n:"Gibibytes (GiB, 1024)", s:"GiB", f:1073741824}, tib:{n:"Tebibytes (TiB, 1024)", s:"TiB", f:
1099511627776} } },
  currency: { label:"Currency", fx:true, units:{
    USD:{n:"US Dollar", s:"USD"}, EUR:{n:"Euro", s:"EUR"}, GBP:{n:"British Pound", s:"GBP"},
    PKR:{n:"Pakistani Rupee", s:"PKR"}, INR:{n:"Indian Rupee", s:"INR"}, BDT:{n:"Bangladeshi Taka", s:"BDT"},
    CNY:{n:"Chinese Yuan", s:"CNY"}, AED:{n:"UAE Dirham", s:"AED"}, SAR:{n:"Saudi Riyal", s:"SAR"},
    QAR:{n:"Qatari Riyal", s:"QAR"}, KWD:{n:"Kuwaiti Dinar", s:"KWD"}, CAD:{n:"Canadian Dollar", s:"CAD"},
    AUD:{n:"Australian Dollar", s:"AUD"}, JPY:{n:"Japanese Yen", s:"JPY"}, CHF:{n:"Swiss Franc", s:"CHF"},
    SGD:{n:"Singapore Dollar", s:"SGD"}, MYR:{n:"Malaysian Ringgit", s:"MYR"}, TRY:{n:"Turkish Lira", s:"TRY"},
    ZAR:{n:"South African Rand", s:"ZAR"}, NGN:{n:"Nigerian Naira", s:"NGN"} } }
};

function tempToC(unit, v){ if(unit==="c") return v; if(unit==="f") return (v-32)*5/9; return v-273.15; }
function tempFromC(unit, v){ if(unit==="c") return v; if(unit==="f") return v*9/5+32; return v+273.15; }

/* fxRates: map code -> units per 1 USD */
function convert(catId, from, to, v, fxRates){
  var cat = CATS[catId];
  if(cat.temp) return tempFromC(to, tempToC(from, v));
  if(cat.fx){
    var rates = fxRates || FX_SNAPSHOT;
```

```
    return (v / rates[from]) * rates[to];
  }
  return v * cat.units[from].f / cat.units[to].f;
}

/* Accept "1,234.5", " 42 ", ".5", "-3", "1e3". Reject everything else. */
function parseInput(raw){
  if(raw === null || raw === undefined) return { ok:false, empty:true };
  var t = String(raw).replace(/[\s,]/g, "");
  if(t === "") return { ok:false, empty:true };
  if(!/^[+-]?(\d+\.?\d*|\.\d+)([eE][+-]?\d+)?$/.test(t)) return { ok:false, empty:false };
  var v = Number(t);
  if(!isFinite(v)) return { ok:false, empty:false };
  return { ok:true, value:v };
}

function fmt(x){
  if(x === 0) return "0";
  if(!isFinite(x)) return "\u2014";
  var ax = Math.abs(x);
  if(ax >= 1e15 || ax < 1e-9) return x.toExponential(6).replace("e+", "e");
  return String(parseFloat(x.toPrecision(10)));
}
```